BACKEND DEVELOPER PORTFOLIO

CONTACT

E. sowork02@gmail.com

GitHub. https://github.com/wishs2

민소원

01	**ABOUT ME**	자기소개
02	**PROJECT**	프로젝트 소개
03	**PROBLEM SOLVING**	문제 해결 경험
04	**RETROSPECTIVE**	프로젝트 회고 및 개선 방향
05	**EDUCATION & CERTIFICATIONS**	교육 및 자격

01 ABOUT ME

BACKEND DEVELOPER

데이터의 정확성과 처리 안정성을 고민하는 Java 백엔드 개발자

Java와 Spring Boot 기반의 리뷰 · 영수증 서비스를 개발하며, 실제 데이터에서 발생하는 오류를 분석하고 데이터 처리 구조와 저장 규칙을 개선해 왔습니다.

KEY ACHIEVEMENTS

01 | OCR 데이터 처리 정확도 개선

46.2% → 97.4% +51.2%p

- Token → Line → Field Extractor 구조 재설계
- 동일 OCR 응답 8건 · 39개 필드 기준 정확도 비교
- 개선 로직을 실제 서비스 파서에 통합하고 동일 데이터셋으로 결과 재검증

02 | 영수증 재사용 및 활성 리뷰 정책 설계

Receipt Reuse · Generated Column · UNIQUE

- 동일 영수증의 중복 INSERT 대신 기존 Receipt ID 재사용
- 회원 · 영수증별 활성 리뷰 1건 제한 및 삭제 후 재작성 허용
- Service · DB 이중 방어와 주요 기능·경계 조건 8개 시나리오 검증

03 | LLM 리뷰 요약 처리 구조 개선

Event → Redis Set → Scheduler

- 변경 업체 ID를 Redis Set에 수집하여 중복 대상 제거
- 동일 처리 주기 내 업체별 요약 작업을 통합하여 반복 LLM 호출을 줄이도록 설계

04 | 리뷰 조회 API 성능 검증

JMeter · 30,000건 요청

- JMeter 기반 리뷰 조회 API 30,000건 요청 테스트
- 응답시간 · 처리량 · 오류율을 분석하고, 부하 조건의 한계를 고려해 측정 결과 해석

02 PROJECT

위치 및 영수증 기반 뷰티샵 리뷰 서비스

>>> Project Overview

기간

- 2025.02 ~ 2025.03
- 개인 리팩토링: 2026.01 ~ 2026.02
- 기능 보완 및 재검증: 2026.10

※ 팀 프로젝트 종료 후 개인 저장소에서 OCR 처리 구조,
영수증 · 리뷰 무결성 및 예외 처리 로직을 개선했습니다.

팀 구성

- 4인 팀 프로젝트

역할

- Backend Developer · Team Lead
· WBS 작성 및 일정 관리
· 업무 분배
· 코드 리뷰 및 Notion 협업

개요

- 영수증 OCR을 통해 결제 정보를 검증하고,
검증된 영수증을 기반으로 뷰티샵 리뷰를 작성할 수 있는 서비스



02 PROJECT

위치 및 영수증 기반 뷰티샵 리뷰 서비스

>>> Project Overview

프로젝트 전체 기술 스택

- Backend
Java 17 · Spring Boot 3 · MyBatis · Spring Security

- Database / Cache
MySQL 8 · Redis

- Frontend
JavaScript · Thymeleaf · Axios

- External API
Naver CLOVA OCR · OpenAI API

- Testing
JUnit · Mockito · Postman · Swagger · JMeter

- Infrastructure
AWS EC2 · AWS RDS · AWS S3 · Docker
※ AWS 인프라 구성 및 배포는 팀원이 담당

- Collaboration
Git · GitHub · Notion · Figma

주요 기능

- 회원 및 인증
  - 회원가입 · 로그인 · 소셜 로그인
  - 이메일/SMS 인증 및 계정 관리

- 뷰티샵 탐색
  - 위치 기반 뷰티샵 조회
  - 뷰티샵 상세 정보 및 사업자 정보 조회
  - 주소 검색 · 지도 기반 위치 조회

- 영수증 인증 및 리뷰
  - OCR 기반 영수증 결제 정보 인증
  - 영수증 기반 리뷰 작성 · 조회
  - LLM 기반 리뷰 요약

- 마이페이지 / 관리자
  - 나의 리뷰 및 회원정보 관리
  - 뷰티샵 등록 · 수정 · 삭제
  - 회원 · 뷰티샵 관리

02 PROJECT

위치 및 영수증 기반 뷰티샵 리뷰 서비스

>>> Project Overview

기여 부분

Backend

- 영수증 OCR
  - Naver CLOVA OCR API 연동
  - 영수증 주요 정보 추출 로직 구현
  - OCR 데이터 파싱 구조 개선

- 리뷰
  - 리뷰 등록 · 조회 · 수정 · 삭제 구현
  - 리뷰 이미지 및 키워드 처리
  - 회원/관리자별 접근 권한 검증

- DB
  - Receipt · Review 및 리뷰 키워드 연관 테이블 설계
  - 식별정보 기반 기존 영수증 조회 · 재사용 로직 구현
  - Generated Column + UNIQUE 제약을 통한 활성 리뷰 중복 방지
  - 소프트 삭제 및 리뷰 재작성 정책 구현 · 검증

- LLM
  - 리뷰 생성 이벤트 기반 요약 대상 수집
  - Redis Set을 활용한 중복 대상 제거
  - Scheduler 기반 리뷰 요약 갱신

Team Lead / UI

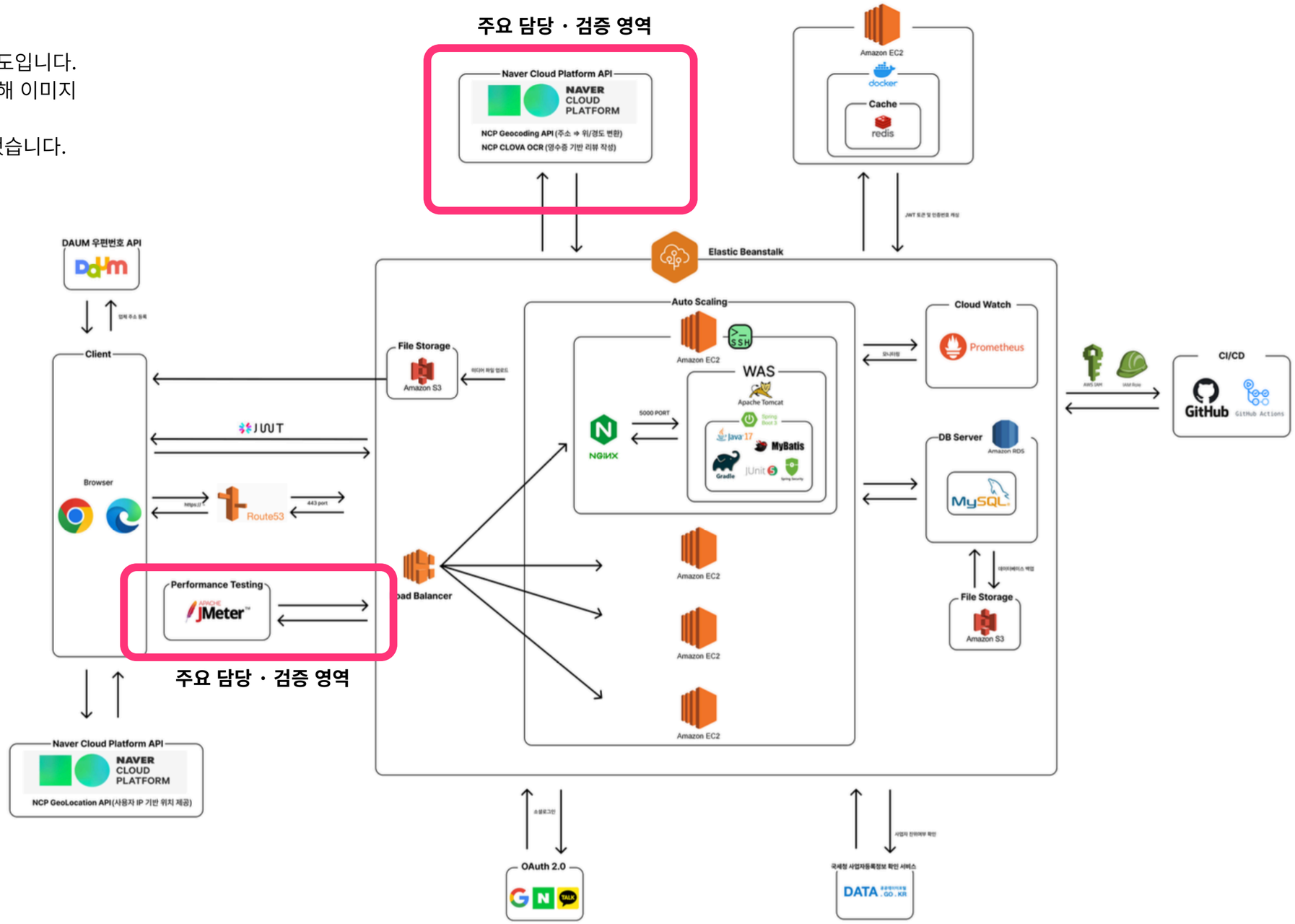
- 협업 및 프로젝트 관리
  - WBS 작성 및 일정 관리
  - 업무 분배 및 진행 상황 관리
  - GitHub Branch · PR · Code Review 기반 협업

- UI
  - Header / Footer 및 주요 화면 Layout 설계·구현

ARCHITECTURE

>>> 프로젝트 구조

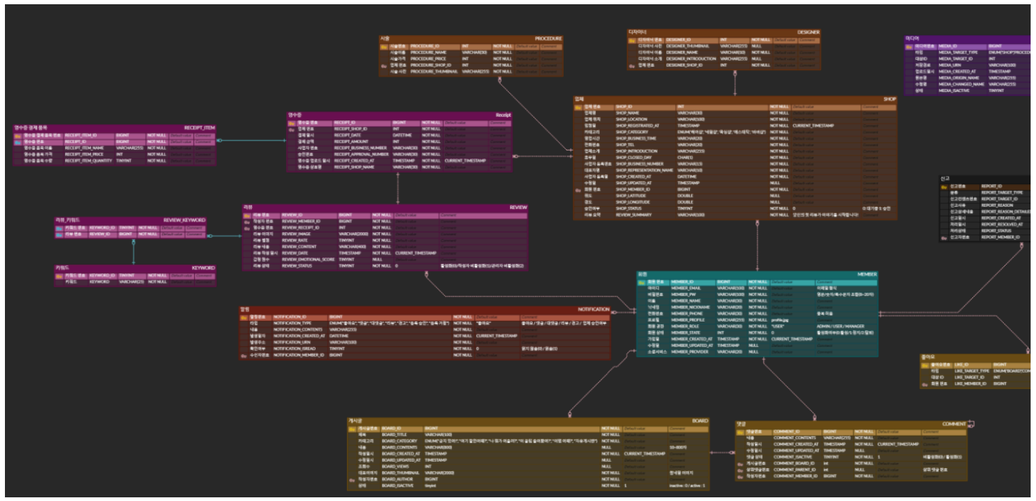
※ 2025년 팀 프로젝트 당시 AWS 기반 시스템 구성도입니다.
이후 개인 포크 환경에서는 독립적인 기능 검증을 위해 이미지 저장 방식을 로컬 파일 시스템으로 변경했으며,
OCR · 리뷰 데이터 처리 로직을 중심으로 리팩토링했습니다.



ERD

>>> Database Design

전체 ERD



RECEIPT & REVIEW | 담당 ERD

최신 MySQL DDL 기준 · 주요 4개 테이블 상세 / MEMBER·SHOP은 외래 키 관계 확인용

MEMBER · 참조

PK	member_id	INT

SHOP · 참조

PK	shop_id	INT

1 : N

REVIEW

PK	review_id	BIGINT
FK	review_member_id	INT
FK	review_receipt_id	BIGINT
	review_image	VARCHAR(300)
	review_rate	TINYINT
	review_content	VARCHAR(400)
	review_date	TIMESTAMP
	review_status	TINYINT
	review_deleted_at	DATETIME
	active_receipt_id	BIGINT / GENERATED

UNIQUE (review_member_id, active_receipt_id)

N : 1

RECEIPT

PK	receipt_id	BIGINT
FK	receipt_shop_id	INT
	receipt_date	DATETIME
	receipt_amount	INT
	receipt_business_number	VARCHAR(255)
	receipt_approval_number	VARCHAR(255)
	receipt_created_at	TIMESTAMP
	receipt_shop_name	VARCHAR(255)

UNIQUE: uq_receipt_identity (5개 식별 컬럼)

1 : N

REVIEW_KEYWORD

PK/FK	keyword_id	TINYINT
PK/FK	review_id	BIGINT

Composite PK (keyword_id, review_id)

N : 1

KEYWORD

PK	keyword_id	TINYINT
	keyword	VARCHAR(25)

Generated Column: active_receipt_id = CASE WHEN review_status = 0 THEN review_receipt_id ELSE NULL END

03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

01. 문제 정의 — 구조 정보가 소실되는 String + Regex 처리

OCR 응답을 하나의 문자열로 변환해 Regex로 탐색하던 구조에서, 영수증 형식에 따라 동일한 필드의 위치와 표현이 달라지며 추출 정확도가 떨어지는 문제를 확인했습니다.

문제 상황

- 초기 구현에서는 빠른 기능 구현을 위해 OCR 응답 전체를 String으로 변환한 뒤 Regex로 필요한 필드를 탐색했습니다.

문제 발생

영수증 형식에 따른 추출 편차

- 동일 필드도 영수증마다 위치·표현 방식이 달라 Regex만으로 대응하기 어려움
- String 변환 과정에서 Token의 위치·Line·Confidence 정보 소실
- 예외 대응 규칙이 하나의 로직에 누적되어 수정 영향 범위 증가

Naver CLOVA OCR
↓
OCR JSON
↓
전체 텍스트 String 변환
↓
Regex 기반 탐색
↓
상호명 · 금액 · 날짜 등 추출

원인 파악

문제의 핵심은 Regex 자체보다, OCR 응답을 단일 문자열로 평탄화하면서 필드 판단에 활용할 수 있는 위치·라인·신뢰도 정보를 잃는 처리 구조에 있었습니다.

핵심 문제

OCR JSON

Token A —— 위치
Token B —— 위치
Token C —— Confidence
Token D —— 위치
　　|
　　| String 변환
　　▼
"XX헤어샵 ... 합계 ... 75,000 ..."

　　↓

위치 정보　　×
라인 관계　　×
Confidence　×

　　↓

Regex가 문자열 표현에 의존

OCR 데이터는 구조화되어 있었지만 추출 과정에서는 비구조화 문자열처럼 처리하고 있었습니다.

03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

02. 구조 재설계 — Token → Line → Field Pipeline

구조화된 OCR 처리 파이프라인으로 재설계

전체 문자열을 직접 탐색하는 방식에서 벗어나
OCRToken → OCRLine → Field Extractor 단계로 처리 구조를 분리했습니다.

OCR의 위치·라인·Confidence 정보를 유지한 채 각 필드가 필요한 문맥을
독립적으로 판단하도록 설계했습니다.

① 구조 정보 보존

OCR JSON → OCRToken → OCRLine

- Text뿐 아니라 위치·Line·Confidence를 보존하여 추출 과정에서 활용

② 필드별 책임 분리

ReceiptFieldExtractor
- ShopNameExtractor
- AmountExtractor
- DateTimeExtractor
- BusinessNumberExtractor
- ApprovalNumberExtractor

- 필드별 규칙을 독립시켜 변경 영향 범위를 제한

③ 후보 기반 판단

Pattern + Keyword + Context + Confidence

- 단일 Regex 일치가 아닌 후보 평가 방식으로 전환

- 각 후보에 Pattern·Keyword·주변 Line·Confidence를
조합하여 점수를 부여하고 최종 후보를 선택했습니다.

개선 후 구조

OCR JSON
↓
OCRTokenMapper
↓
OCRToken
↓
OCRLineGrouper
↓
OCRLine
↓
OCRReceiptParser
↓
ReceiptFieldExtractor
- ShopNameExtractor
- AmountExtractor
- DateTimeExtractor
- BusinessNumberExtractor
- ApprovalNumberExtractor

↓
ExtractedField<T>

03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

03. 추출 전략 — Pattern에서 Context 기반 후보 평가로

문자열 매칭에서 문맥 기반 필드 추출로

영수증마다 필드의 위치와 표현이 달라지는 문제에 대응하기 위해, 각 Extractor가 **패턴·키워드·주변 Line·Confidence를 조합하여 후보를 평가**하도록 개선했습니다.

공통 결과 모델 — ExtractedField

Value + Confidence + SourceText

- 추출값과 판단 근거를 함께 보존하여 검증 및 실패 분석에 활용

① 결제금액

숫자 후보
↓
TOTAL Keyword 가산
↓
SKIP Keyword 제외
↓
최종 후보 선택

- 숫자 패턴 자체가 아니라 **금액 주변의 의미를 함께 평가**

② 결제일시

Date Pattern
↓
동일 Line Time 탐색
↓
인접 Line 보완
↓
거래/승인 Keyword 가산

- 날짜와 시간이 서로 다른 OCR Line으로 분리되는 경우까지 고려하여 인접 문맥을 탐색

③ 상호명

상호 후보
↓
상호 Keyword 가산
↓
업무 Keyword 제외
↓
주소 오인식 검증

- 짧은 한글 상호가 주소 패턴의 일부로 오인되어 제외되는 케이스를 확인하고, 주소 판별 조건과 상호 후보 검증 규칙의 경계를 조정

03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

04. 정확도 검증 — 동일 데이터셋 Before / After Test

Before / After 테스트를 통한 추출 정확도 검증

동일한 8개 영수증 OCR 응답에 기존 로직과 개선 로직을 각각 적용하고, Ground Truth와 비교하여 필드별 추출 정확도를 측정했습니다.

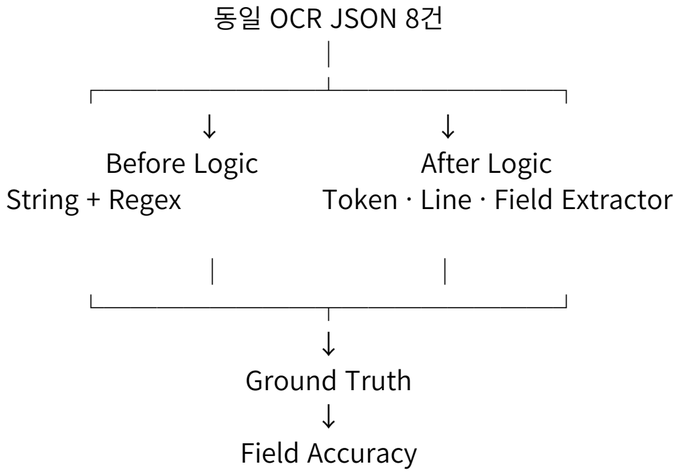
① 테스트 설계

Dataset
영수증 8건

Evaluation Fields
상호명 · 결제금액 · 결제일시 · 사업자번호 · 승인번호

Total Evaluations
39 Cases

Evaluation



※ 사업자번호 1건은 Ground Truth 평가 대상에서 제외하여 총 39개 필드를 비교
※ 개선 로직의 실제 서비스 파서 통합 후 동일 OCR JSON 8건·39개 필드 기준 회귀 검증

② Overall Accuracy

46.2% → 97.4%
+51.2%p

③ Field Accuracy

Field	Before	After	변화
상호명	25.0%	**87.5%**	+62.5%p
결제금액	12.5%	**100.0%**	+87.5%p
결제일시	100.0%	**100.0%**	—
사업자번호	71.4%	**100.0%**	+28.6%p
승인번호	25.0%	**100.0%**	+75.0%p
TOTAL	**46.2%**	**97.4%**	+51.2%p

※ 기존 평가 데이터셋에서의 개선 결과이며, 새로운 영수증 형식에 대한 일반화 성능은 추가 검증이 필요합니다.

03 PROBLEM SOLVING

>>> ① OCR 데이터 처리 구조 개선

05. 결과 분석 — 개선 효과와 Remaining Case

④ Result Analysis

01. 결제금액 | 12.5% → 100%

Before
숫자 Pattern만으로 후보를 탐색하여 공급가액·사업자번호 등 다른 숫자를 오인식

After
TOTAL/SKIP Keyword + 주변 Line 문맥을 후보 평가에 반영

Result
8건 모두 결제금액을 정확히 추출

02. 숫자 필드 분리 | 승인 25→100 / 사업자 71.4→100

Before
여러 숫자 필드가 하나의 처리 로직에서 경쟁

After
BusinessNumber / ApprovalNumber Extractor로 책임 분리

Result
각 숫자의 의미를 독립적으로 판별하여 오인식 제거

03. 상호명 | 25→87.5

Before
승인금액, 영수증, 주소 일부 등이 상호 후보로 선택

After
상호 Keyword + 제외 Keyword + Line 문맥 적용

Result
8건 중 7건 정확 추출

⑤ Remaining Case

OCR 원문 인식 오류

Ground Truth
CJ올리브영(주)OO점

↓ OCR
CJ올리브영(주) XX광역시 YY구 OO점 xx로 123 ...

↓ Extractor
CJ올리브영(주)

- 최종 1건은 OCR 단계에서 상호명 일부가 주소와 혼합되고 점이 다른 문자로 인식되어 Extractor만으로 원래 상호명을 완전히 복원하기 어려웠습니다.

- 후처리 규칙을 계속 추가하기보다 OCR Token 좌표 기반 영역 분리 또는 매장 데이터와의 검증을 추가 개선 방향으로 판단했습니다.

결과

동일한 8개 영수증의 39개 평가 필드에서 추출 정확도를 **46.2% → 97.4% (+51.2%p)**로 개선했습니다.

남은 오류는 후처리 규칙을 추가하기보다 OCR 입력 단계의 구조적 한계로 분리하여 후속 개선 방향을 정의했습니다.

03 PROBLEM SOLVING

>>> ② 영수증 재사용 및 활성 리뷰 정책 설계

01. 중복 범위 재정의 — 영수증 재사용과 활성 리뷰 제한

문제

① 영수증 중복 차단으로 인한 재작성 불가

동일 영수증의 중복 INSERT를 차단했지만, 리뷰 삭제 이후에도 원본 영수증이 DB에 남아 있어 재인증 시 중복 오류가 발생했습니다.

② 리뷰 상태를 고려하지 않은 중복 기준

리뷰의 삭제 상태를 고려하지 않은 중복 판단으로, 삭제 이후에도 같은 영수증의 재작성이 제한되었습니다.

요구사항

Receipt 동일 식별정보 → 기존 영수증 재사용
Review 동일 회원 · 영수증 → 활성 리뷰 1건 제한
Delete 원본 이력 보존 → 삭제 후 재작성 허용
Member 서로 다른 회원 → 동일 영수증 사용 허용

해결 방향

Application
중복 규칙 정의
↓
Database
UNIQUE Constraint
↓
Result
저장 단계 무결성 보장

무결성 규칙

대상	식별 기준	DB 제약
Receipt	사업자번호 + 승인번호 + 결제일시 + 결제금액 + 매장명	Composite UNIQUE
Review	회원 ID + 활성 영수증 ID	Generated Column + UNIQUE

정책 적용 결과

Receipt
동일 영수증 존재?

YES → 기존 ID 재사용
NO → 신규 INSERT

Review
활성 리뷰 존재?

YES → 중복 차단
NO → 작성 허용

삭제된 리뷰는 이력을 보존하고 재작성 허용

중복 영수증의 저장은 제한하되 기존 데이터를 재사용하고,
리뷰의 중복은 회원과 활성 상태를 기준으로 판단하도록 정책을 분리했습니다.

03 PROBLEM SOLVING

>>> ② DB 제약 기반 무결성 보장 및 예외 처리

02. 무결성 보장 — Service · DB 이중 방어와 예외 분리

영수증 저장 로직

01 Receipt — 영수증 재사용
식별정보 기반 조회 → 기존 ID 재사용 / 미존재 시 신규 저장

02 Review — 활성 리뷰 중복 방지
Service 사전 조회 → 리뷰 INSERT → DB UNIQUE 제약

03 Soft Delete — 데이터 보존
review_status 변경 → 원본 리뷰·키워드·이미지 이력 유지

예외 처리

01 DB 제약 충돌 감지
리뷰 INSERT 중 UNIQUE 제약 충돌 시
DataIntegrityViolationException 처리

02 대상 오류 판별
MySQL Error Code 1062, SQLState 23000,
UNIQUE 인덱스명 확인

03 비즈니스 예외 변환
활성 리뷰 중복 위반만
DUPLICATE_REVIEW(RV008)로 변환

예외 처리

Service에서 활성 리뷰 중복 여부를 사전에 검사하고, DB UNIQUE 제약을
최종 방어선으로 적용했습니다.

DB 무결성 예외를 일괄적으로 중복 리뷰 오류로 처리하지 않고, **대상 UNIQUE 제약 위반에 한해서만 RV008로 변환**하여 다른 무결성 오류의 원인을 보존했습니다.

```
if (duplicateKey && targetConstraint) return true;
```

중복 키 오류와 대상 UNIQUE 인덱스를 함께 확인해 예외 변환 여부를 판별

주요 테스트(8개 시나리오 중 5개 발췌)	결과
동일 회원 · 영수증 활성 리뷰 중복	차단
리뷰 삭제 후 동일 영수증 재작성	성공
다른 회원의 동일 영수증 사용	성공
삭제된 원본 데이터 보존	확인
반복 삭제 · 재작성	성공

결과

영수증 재사용과 활성 리뷰 중복 제한을 분리하고, **데이터 무결성 · 이력 보존 · 삭제 후 재작성 정책**이 정상 동작함을 검증했습니다.

03 PROBLEM SOLVING

>>> ③ LLM 리뷰 요약 호출 최적화

01. 반복 호출 문제 — 호출 단위 재설계

Review A ──→ LLM ──→ Summary Update
Review B ──→ LLM ──→ Summary Update
Review C ──→ LLM ──→ Summary Update
⋮

동일 업체에 대한 리뷰가 연속해서 작성될 경우 반복적인 요약 요청 발생

문제

리뷰 작성 이벤트마다 요약을 갱신하여, 동일 업체에 여러 리뷰가 연속 등록될 경우
LLM API 호출이 반복되는 구조였습니다.

해결 방향

리뷰 작성 이벤트에서 즉시 LLM을 호출하지 않고, 변경된 업체 ID를 Redis Set에
수집한 뒤 일정 주기로 일괄 처리하도록 변경했습니다.

Review A → Shop 10 ┐
Review B → Shop 10 ├→ Redis Set {10,20}
Review C → Shop 20 ┤ ↓
Review D → Shop 10 ┘ Scheduler
↓
업체 단위 처리 — **리뷰 단위 처리에서 변경 업체 단위 처리로 전환**

Redis Set 선택 이유

업체 ID의 중복을 자동 제거하여 동일 업체의 여러 변경 이벤트를 하나의 처리 대상으로 통합

02. Event → Redis → Scheduler 기반 처리 구조

전체 처리 흐름

Review 저장
↓
Event 발행
↓
Async Listener
↓
Redis Set {shopId} — 중복 Shop ID 자동 제거
↓
Scheduler
↓
변경 Shop ID 조회
↓
Review 조회
↓
LLM Summary
↓
review_summary 갱신

03 PROBLEM SOLVING

>>> ③ LLM 리뷰 요약 호출 최적화

역할 분리

① EVENT

리뷰 변경 사실 전달

---

② ASYNC LISTENER

요약 대상 수집을 리뷰 저장 흐름에서 분리

---

③ REDIS SET

Shop ID 저장 + 중복 제거

---

④ SCHEDULER

수집된 업체의 요약 작업 실행

---

⑤ LLM SERVICE

리뷰 기반 요약 생성

- FAILURE HANDLING

업체 단위 예외 처리로 하나의 요약 실패가
이후 업체 처리까지 중단시키지 않도록 구성

03. 개선 결과와 Trade-off

구분	Before	After
처리 기준	리뷰 이벤트	변경 업체
대상 관리	개별 이벤트 처리	Redis Set 으로 변경 업체 수집
동일 업체 변경	반복 처리 가능	Shop ID 통합
처리 시점	이벤트 발생 시	Scheduler 주기 처리
처리 범위	이벤트마다 요약	변경 업체만 요약

결과

- 리뷰 이벤트마다 요약 API를 호출하던 구조를 변경 업체 단위의 주기적 처리로 전환했습니다.
- Redis Set으로 동일 처리 주기 내 중복 Shop ID를 제거하여 불필요한 요약 호출을 줄일 수 있도록 설계했습니다.

Trade-off

호출 효율 ↔ 요약 최신성

호출 중복을 줄이는 대신 최신 리뷰가 요약 결과에 반영되기까지 지연이 발생할 수 있습니다.

Further Improvement

실제 토큰 사용량에 따른 비용 절감 효과와 실패 작업의 재처리는 향후 검증 · 개선 과제로 구분했습니다.

03 PROBLEM SOLVING

>>> ④ 리뷰 조회 API 부하 테스트 및 측정 결과 분석

문제

이미지 파일의 저장·제공 책임이 애플리케이션 서버에 함께 존재

Application Server
├ 리뷰 데이터 처리
└ 이미지 파일 저장·제공

구조 변경 및 측정 방식

기존 애플리케이션 서버의 로컬 이미지 경로를 사용하는 구조에서 S3 이미지 주소를 참조하는 구조로 변경한 전후의 리뷰 조회 API 성능을 비교했습니다.

JMeter로 각 환경에 30,000건의 요청을 수행하고 응답시간, 처리량, 오류율을 측정했습니다.

Before

Application Server
├ 리뷰 데이터
└ 이미지 파일 저장·제공

After

Application Server
└ 리뷰 데이터 + 이미지 참조 정보
↓
S3
이미지 저장·제공

S3 선택 이유

- 이미지 저장 · 제공 책임 분리
- 서비스에서는 이미지 참조 정보 중심으로 관리

테스트 조건/환경

Target 업체 리뷰 목록 조회 API
Tool Apache JMeter
Requests Before / After 각각 30,000건
Environment Ryzen 5 3400G · RAM 32GB · Windows 11 · MySQL
Role 부하 테스트 수행 · 결과 분석 · 테스트 보고서 작성

부하 테스트 측정 결과 및 해석

지표	Before	After
평균 응답시간	약 2,000 ms	956 ms
중앙값	2,061 ms	935 ms
P90	2,406 ms	1,166 ms
P95	2,528 ms	1,255 ms
P99	2,753 ms	1,392 ms
오류율	0%	0%
Throughput	약 159 req/s	약 1,004 req/s

JMeter를 사용하여 구조 변경 전후 리뷰 조회 API에 각각 30,000건의 요청을 수행하고, 측정 결과를 분석하여 테스트 보고서를 작성했습니다.

기록된 평균 응답시간은 약 2,000ms에서 956ms로, Throughput은 약 159req/s에서 1,004req/s로 차이를 보였습니다.

이미지 저장 · 제공 위치를 로컬 서버에서 S3로 분리한 것이 성능 차이에 영향을 주었을 가능성이 있다고 판단했습니다.

다만 동시 사용자 수와 Ramp-up 등 세부 부하 조건이 남아 있지 않아, 측정 차이를 S3 전환의 직접적인 성능 개선 효과로 단정하지 않았습니다.

이를 통해 성능 테스트에서는 요청 수뿐 아니라 동시성 · 부하 패턴 · 실행 환경을 함께 기록해야 한다는 점을 확인했습니다.

03 PROBLEM SOLVING

>>> ⑤ Testing & Collaboration

Testing

1. API / Controller Test

- Postman · Swagger · MockMvc
- 주요 CRUD API 및 응답 형식 검증
- Mockito 기반 OCR 업로드·리뷰 요약 API 의존성 분리

2. Unit / Regression Test

- JUnit · Mockito
- 리뷰 요약 서비스의 정상·빈 데이터 분기 테스트 작성
- 실제 서비스 OCR 파서 회귀 검증: 38/39개 필드 일치(97.4%)

3. Performance Test

- JMeter · Before / After 각각 30,000건 요청
- 리뷰 조회 API의 응답시간·Throughput·오류율 분석
- 테스트 수행 및 결과 보고서 작성, 측정 조건의 한계 분석

Collaboration

1. WBS & Schedule

- WBS 기반 기능별 업무 배정 및 일정·진행 상황 관리

2. GitHub Collaboration

- Branch · Issue · PR · Code Review를 통한 변경사항 관리

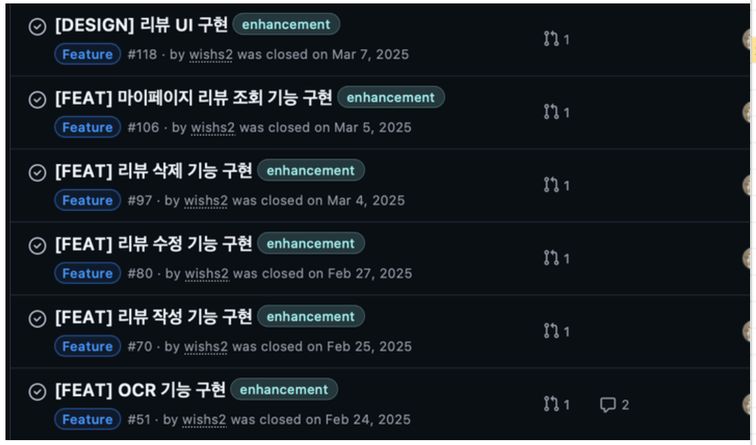
3. Documentation

- 요구사항 · API 명세 · 작업 범위 문서화

4. Team Lead

- 기능별 작업 조율 및 프로젝트 진행·발표 관리

GitHub Issues



WBS

1. 주제선정 및 피드백	2025-02-11	2025-02-17	100%	
1.1 아이디어 산출 및 구체화	2025-02-11	2025-02-14	완료	공통
1.2 피드백	2025-02-17	2025-02-17	완료	공통
1.3 피드백 내용 보완	2025-02-17	2025-02-18	완료	공통
2. 분석 / 설계	2025-02-11	2025-02-17	100%	
2.1 역할 구성	2025-0	MSW_002		
2.2 요구사항정의	2025-0	MSW_003		
2.3 문서 작성	2025-0	MSW_004		
2.3.1 기획서 작성	2025-0	MSW_005		
2.3.2 요구사항 정의서 작성	2025-0	MSW_006		
2.3.3 WBS 작성	2025-02-1	MSW_007	완료	공통
2.3.4 FLOW CHART 작성	2025-02-12	2025-02-14	완료	공통
2.3.5 애플리케이션 아키텍처 설계	2025-02-12	2025-02-12	완료	공통
2.3.6 UseCase 작성			보류	공통
2.3.7 프로젝트 정의서	2025-02-1	2025-02-14	완료	공통
2.3.8 API 정의서 작성	2025-02-1	2025-02-16	완료	공통
2.3.9 수행일지 작성				공통
2.4 DB설계	2025-02-12	2025-02-13	완료	공통
2.5 와이어프레임 작성	2025-02-11	2025-02-19	완료	공통
2.6 UI/UX 설계	2025-02-11	2025-02-19	완료	공통

리뷰 작성

Requirements

	장당 30MB 이내
필터링	금칙어 필터링
리뷰 유효기간	영수증에서 추출한 데이터 중 결제 일시를 활용하여 방문 후 30일 이내로만 리뷰 작성 가능
별점	업체에 별점⭐ 매기기
키워드	작성자가 키워드 리뷰 작성 시 키워드 선택 이후 리뷰 조회 시 키워드를 통해 검색
영수증 리뷰 ***	OCR API를 이용한 리뷰 작성
리뷰 유효성 검증	결제 금액 검증, 결제 일자 검증, 상호명 영수증 증명 -> 이미 사용한 영수증인지 검증 DB 테이블에 저장

04 RETROSPECTIVE

프로젝트 회고 및 개선 방향

프로젝트를 통해 배운 점

01. 설계
빠른 구현보다 변경 가능한 구조를 고려하게 되었습니다.
OCR 데이터를 구조화하고 역할별 책임을 분리하면서 유지보수성을 고려한 설계의 중요성을 경험했습니다.

02. 데이터
데이터 무결성은 저장 단계부터 설계해야 한다는 것을 배웠습니다.
비즈니스 규칙을 DB 제약조건으로 구현하며 데이터 자체의 신뢰성을 확보하는 방법을 경험했습니다.

03. 외부 API
외부 API는 호출 횟수와 처리 시점까지 고려해야 한다는 것을 배웠습니다.
LLM 호출을 이벤트와 Scheduler 기반으로 분리하여 불필요한 반복 처리를 줄이는 구조를 경험했습니다.

04. 검증
구현 결과를 테스트하고 측정 조건까지 기록하는 중요성을 배웠습니다.
API·부하 테스트를 통해 기능과 성능을 확인하고, 재현 가능한 테스트 조건의 중요성도 경험했습니다.

향후 개선 방향

영역	현재 구현	향후 개선
OCR	Token → Line → Field Extractor 기반 구조화 및 서비스 파서 회귀 검증 (38/39개 일치)	미지 영수증 테스트 확대, OCR 원문 인식 오류 대응, 추출 규칙·신뢰도 기준 정교화
Data Integrity	영수증 재사용, 활성 리뷰 UNIQUE 제약, 소프트 삭제 및 예외 분리	동시 요청 충돌 테스트, 영수증 식별 기준 정교화, 자동화 테스트 확대
LLM	Event → Redis Set → Scheduler 기반 업체별 요약 처리	실패 작업 재처리, 스케줄러 안정성 및 호출량 계측
운영	팀 배포 경험 및 로컬 기능 검증	직접 배포, CI/CD 구축, 로그 · 모니터링 경험 확장

앞으로의 개발 방향

기능 구현 이후에도 OCR 추출 구조, 영수증·리뷰 데이터 무결성, LLM 호출 흐름을 개선하며 변경에 대응할 수 있는 설계의 중요성을 경험했습니다.

앞으로는 자동화된 회귀 테스트와 실제 운영 환경으로 검증 범위를 확장하여, 구현 이후의 안정성과 유지보수성까지 고려하는 백엔드 개발자로 성장하고자 합니다.

05 EDUCATION & CERTIFICATIONS

EDUCATION & TRAINING

신안산대학교

- 2021년 3월 - 2023년 2월
- 호텔경영과 (4.28/4.5)

신세계아이앤씨

- 2024년 9월 - 2025년 3월
- JAVA 기반 백엔드 개발자 양성과정

- 미니 팀 프로젝트 및 최종 프로젝트 수행

CERTIFICATIONS

정보처리산업기사

- 2025년 6월 취득

SQLD

- 2026년 6월 취득

THANK YOU